Easy Campus Stay deployment report

Group 22 | SEZG583 Assignment 2 | 31 October 2026

Easy Campus Stay continues Assignment 1 using Python, PHP and MariaDB. The deployment files support Docker Compose and an optional Minikube extension. This report records the design, verified reference checks and team execution evidence still to complete.

1 Application and container architecture

Easy Campus Stay is a simple room-booking system for campus visitors. Inventory manages room stock. Reservation coordinates a durable booking workflow. Payment and Notification are mocks; no real financial transaction or email occurs. The PHP browser interface is an API client with no direct database access.

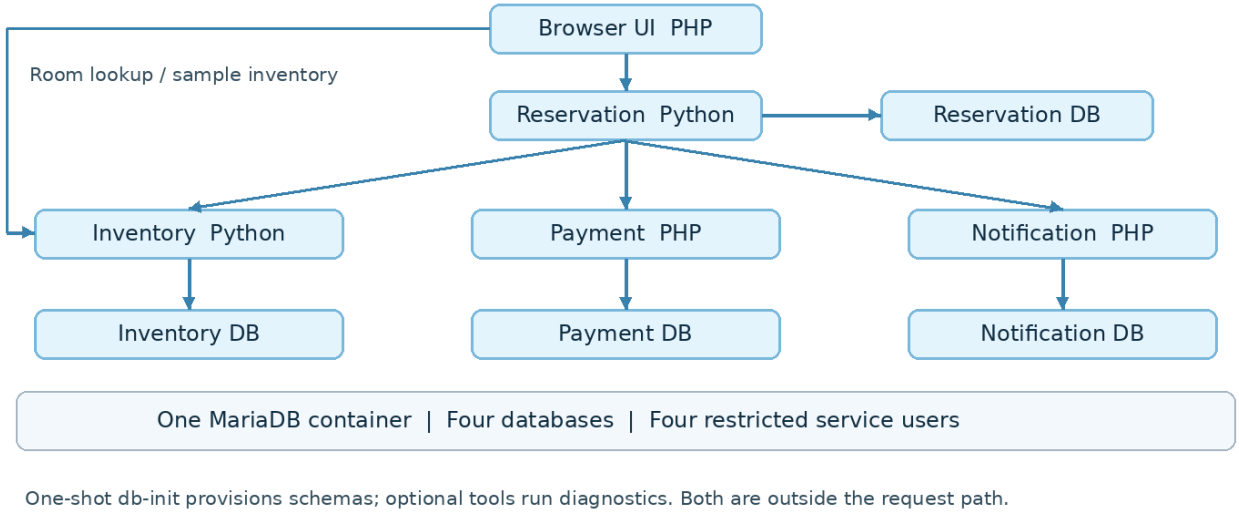


Figure 1. Booking REST calls and logical database ownership. One MariaDB engine hosts four databases; each application receives only its own database login. Technical campusstay names remain for compatibility.

Runtime component	Deployment role
Inventory and Reservation	Separate Python 3.12 images with Uvicorn
Payment, Notification and UI	Separate PHP 8.3 Apache images
MariaDB 10.11	Persistent engine with four restricted databases
db-init and tools	Setup and diagnostics; not application request handlers

Scope: Docker Compose is the core deployment. Minikube, manual Inventory scaling and recovery after pod deletion implement the optional extension. Runtime evidence for Docker and Kubernetes must still be generated on the team machine.

2 Containerization and Docker Compose

Each of the four microservices has its own Dockerfile, and the browser UI has a fifth. Python images install the pinned service dependencies, copy only runtime source and run as an application user. PHP images use Apache with PDO MySQL and cURL; only public files are served, with API routes falling back to index.php. PHP OpenAPI files are included in their images.

Two support images provide one-shot database provisioning and the original API challenge runner. They are separate from application images. MariaDB uses an upstream image. Images copy explicit files; .dockerignore excludes local configuration, secrets, runtime files and evidence.

Configuration and readiness

Compose reads a private root .env generated locally. The repository contains only .env.example placeholders. Credentials are supplied at runtime, not baked into images. Each application receives its own service password; the administrator password is limited to MariaDB and provisioning. The optional test container has the four service credentials to assert database isolation.

Inside the network, Reservation calls http://inventory:8080, http://payment:8080 and http://notification:8080. localhost would point back to Reservation. The UI has an explicit allow-list for these service DNS names. The database is not published to the host. UI and API ports bind only to host loopback.

MariaDB health is checked before db-init runs. Application services wait for successful provisioning; Reservation and UI wait for healthy dependencies. db-init Exited (0) is expected. API health endpoints check each service database. UI health checks its process; a successful booking proves downstream communication.

Reproducible startup and restart

Use C:\Projects\campusstay_assignment2. If deploy/configure.py is missing, first preview and import the deployment additions from the separate package using deploy/import_assignment2.py. The helper preserves business source and private configuration and stops on incompatible patches.

```
py -3.12 deploy/configure.py
docker compose config --quiet
docker compose up --build
```

Use a second terminal to inspect docker compose ps -a and logs, then book through http://127.0.0.1:8180/. The optional tools container runs e2e, concurrency, idempotency, saga and 12 isolation assertions, saving real output to the evidence bind mount.

```
docker compose --profile test run --rm --build tools
docker compose down
docker compose up -d
```

The database_data named volume survives down without -v. Restart validation must retrieve a booking created before shutdown. A fresh booking after restart alone is insufficient evidence of persistence. Retain the same private configuration for the existing volume.

3 Deployment troubleshooting and situated learning

Configuration defect identified and corrected

Review found a concrete compatibility defect in ui/bootstrap.php: ui_settings only overrode keys already loaded from a local .env. In an environment-only container setup with no .env in the image, service URLs were absent. The proxy also rejected every host except localhost and 127.0.0.1, preventing Compose service DNS addresses.

The fix reads the four known URL variables directly from the environment and supports an explicitly configured UI_ALLOWED_UPSTREAM_HOSTS list. The default remains loopback-only for the original local setup. Credentials in URLs are rejected. Native PHP regression checks verified environment-only configuration and rejection of an unapproved host.

Actual Docker incident evidence remains to be completed

The reported missing deploy/configure.py error is a Windows setup issue before Docker starts. The updated guide checks the checkout path and imports the deployment files. This setup issue alone does not satisfy the required container deployment incident.

The authoring environment had no Docker daemon or container privileges. Therefore, the configuration defect above is a verified pre-deployment finding, not a claimed incident observed inside a running Docker deployment. To satisfy Task 3, the team must record one genuine deployment problem encountered during its own run using the supplied incident worksheet.

Required evidence	Team completion
Problem and symptom	Record failing command/UI action, time and observed error.
Diagnosis	Save container status, relevant logs and targeted configuration/DNS checks.
Solution and validation	Record the actual change; repeat the affected workflow and show success.
Learning reflection	Explain why the failure occurred and what check would prevent it.

Useful diagnosis commands

```
docker compose ps -a
docker compose logs --tail=100 reservation
docker compose exec reservation python -c `
  "import socket; print(socket.gethostbyname('inventory'))"
```

A controlled localhost override is included only for rehearsal. It deliberately sends Inventory calls to Reservation loopback port 8101, causing booking progress to fail or remain pending. Removing the override and recreating Reservation restores its configured service URL. This exercise must be labelled as injected; it is not evidence of a naturally encountered incident.

Learning: local health is narrower than end-to-end health. Correct DNS names, internal ports and readiness checks matter when moving a working local application into containers. Do not expose .env, full Secret output or environment dumps while diagnosing.

4 Minikube scaling and self healing extension

The optional deployment uses the same application images in a local Minikube cluster. Declarative resources include a namespace, MariaDB Deployment with persistent storage, database provisioning Job, five application Deployments and ClusterIP Services, and a diagnostic Pod. The deployment helper explicitly targets the minikube context.

Deployment process

The reviewed PowerShell helper builds and loads the project images into Minikube, creates a Secret in memory from the private local configuration, waits for MariaDB, runs provisioning and waits for application readiness. Secret data is not saved as a repository manifest. Kubernetes credentials are not encrypted merely because they are represented as a Secret.

```
powershell -NoProfile -ExecutionPolicy Bypass `
  -File deploy/windows/Deploy-Minikube.ps1
kubectl --context=minikube -n easy-campus-stay get pods,jobs,services,pvc
```

Stop Compose before this phase on the 8 GB practice laptop. The UI is exposed locally with kubectl port-forward service/ui 8180:8080 in a dedicated terminal. A new booking must succeed through this Kubernetes route; Running pods alone do not satisfy the advanced requirement. Compose and Minikube use separate data volumes.

Scaling and replacement demonstration

Demonstrate-SelfHealing.ps1 requests three Inventory replicas, records their names and UIDs, deletes one Inventory pod and waits for three Ready replicas again. It checks that the deleted UID is absent and writes before/after evidence. A further booking and the cluster workflow suite validate the system after replacement.

```
powershell -NoProfile -ExecutionPolicy Bypass `
  -File deploy/windows/Demonstrate-SelfHealing.ps1
powershell -NoProfile -ExecutionPolicy Bypass `
  -File deploy/windows/Check-Minikube.ps1
```

Readiness probes call /health to include local database availability. TCP liveness probes check a listening process without repeatedly restarting every application during a shared database outage. MariaDB uses a single replica and Recreate updates; this is not a highly available database design.

Evidence and practical limits

Required team outputs: cluster resources and image identities; successful Kubernetes booking; three Inventory replicas; deleted/replacement pod UIDs; a successful workflow afterward. These runtime checks were not executable in the authoring environment and remain pending. This extension demonstrates manual scaling and Deployment reconciliation, not HPA, rolling rollback, distributed tracing or production availability.

5 Validation contributions and disclosure

Check	Actual result
Python unit and configuration cases	35 passed on native Linux
Native PHP Payment / Notification cases	10 / 4 passed
Original API flow and three challenges	All four scripts passed on native Linux
Cross-database access attempts	All 12 denied by restricted accounts
UI configuration regression and PHP lint	Passed
Compose 2.39.4 configuration validation	Passed; no Docker daemon execution
Kubernetes 1.34.1 official OpenAPI schema	16 resources passed static validation
PowerShell 7.4.7 syntax parsing	Four helper scripts passed; Windows execution pending
Deployment import safeguard cases	9 passed on Linux; Windows execution pending
Docker build / restart and Minikube recovery	Team runtime evidence pending

Native checks used real Python, PHP and MariaDB processes. They are reference evidence, not container deployment evidence. The package records this distinction under evidence/reference_native_linux. The supplied source is the last provided Assignment 1 baseline, not an authenticated copy of the latest GitHub main branch.

Individual contribution allocation

Member	Assignment 2 review and verification responsibility
Ramyavarshini P. 2025tm12121	Inventory image, concurrency, scaling/replacement evidence
Mujaffar Abdul Dange 2025tm93137	Reservation/UI configuration, integration and screen operation
Yash Yogesh Moharil 2025tm93172	Payment image, deployment diagnosis and payment workflow
Ambuj Saxena 2025tm12053	Notification image, health, restart and retry verification

These are assigned responsibilities. Each member must add actual changes, verification and genuine Assignment 2 commit references after doing the work. No personal achievements or commit hashes are invented. Complete the actual incident record and Windows/cluster results before final submission.

AI use and submission links

Substantial AI assistance prepared deployment code, configuration fixes, documentation, test orchestration and this report. The team must independently review and explain the files, execute the deployment, record real evidence and describe its actual contributions. Add the team’s specific tool use and verification details.

Repository: https://github.com/2025tm93137-cyber/EasyCampusStay---Room-Booking-System---microservices
Assignment 2 video URL and safe evidence/Drive URL: add after the team recording and upload.

Technical references: Docker Compose startup order; Kubernetes probes; Minikube start/image documentation. Full official URLs are listed in README_ASSIGNMENT2.md. The attached institution manual governs submission: report maximum five main pages; video approximately six to eight minutes.

Appendix A Team Windows execution record

Complete these fields from the actual Windows deployment before submission. Attach safe evidence links and replace pending entries when verified. This appendix is outside the five-page main report.

Item	Actual team record
Execution date and operator	To complete after the Windows run
Windows edition and build	To complete from winver
Docker Desktop and Compose versions	To complete from actual command output
Compose build health and workflow result	To complete with evidence path and observed result
Persistence check	Earlier booking ID and post-restart lookup result
Genuine deployment incident	Symptom, cause, diagnostic evidence, actual fix and retest
Minikube extension if attempted	Versions, working booking, replica/UID evidence and result
Repository	https://github.com/2025tm93137-cyber/EasyCampusStay---Room-Booking-System---microservices
Video and safe evidence links	To complete after upload and access check

Actual Assignment 2 member contributions

Member	Actual files and verification	Commit references
Ramyavarshini P.	Enter actual Inventory files reviewed and test results	Add real Assignment 2 commit IDs
Mujaffar Abdul Dange	Enter actual Reservation and UI files reviewed and test results	Add real Assignment 2 commit IDs
Yash Yogesh Moharil	Enter actual Payment and diagnosis work and test results	Add real Assignment 2 commit IDs
Ambuj Saxena	Enter actual Notification and restart checks and results	Add real Assignment 2 commit IDs

Use git log --oneline or GitHub Commits to obtain real references. Assignment 1 hashes do not prove Assignment 2 work. Each member should describe only work personally reviewed or performed.

Team AI use and verification

Add the tools used, dates, files reviewed, tests personally executed and corrections made. Retain the declaration of substantial AI assistance in the main report.